Project Report: Student Academic Record Management System

Chapter 1: Introduction

In the contemporary educational landscape, the efficient management of student academic records is paramount for colleges and universities. Traditional paper-based systems are often cumbersome, prone to errors, and inefficient for data retrieval and analysis. The need for digital record-keeping has become increasingly evident to streamline administrative processes, enhance data accuracy, and provide quick access to student information for faculty, administration, and students themselves. Digital systems facilitate better decision-making, improve communication, and ensure the long-term integrity and security of sensitive academic data.

This project aims to address these challenges by developing a lightweight **Student Academic Record Management System (SARMS)**. The primary objective is to build a foundational tool using core Python programming concepts that can efficiently store, retrieve, update, and delete student academic records. This system will demonstrate the practical application of fundamental programming principles to create a functional and user-friendly solution for managing academic data.

Chapter 2: Python Concepts Used

This project leverages two fundamental Python concepts: **Dictionaries** and **File Handling**. These concepts are crucial for structuring and persisting the student academic data.

Dictionaries (Key-Value Pairs)

A **dictionary** in Python is an unordered collection of data values, used to store data values like a map, which, unlike other data types that hold only a single value as an element, holds `key:value` pairs. Each `key` must be unique and immutable (e.g.,

strings, numbers, or tuples), while `values` can be of any data type and can be duplicated. Dictionaries are highly optimized for retrieving values when the key is known, making them ideal for scenarios where data needs to be accessed quickly based on a unique identifier.

In the SARMS project, a dictionary is used to store individual student records. The student's unique **Roll Number** serves as the `key`, and the `value` associated with each key is another dictionary containing the student's details such as 'Name', 'Course', and 'Grade'. This nested dictionary structure allows for organized and efficient storage and retrieval of each student's comprehensive academic profile. For example, `students['101']` would return a dictionary like `{'Name': 'John Doe', 'Course': 'Computer Science', 'Grade': 'A'}`.

File Handling (open(), read/write modes)

File handling in Python refers to the process of reading from or writing to files on a computer's storage. It allows programs to interact with external data sources, ensuring data persistence beyond the program's execution. Python provides built-in functions and methods to perform various file operations, including opening, reading, writing, and closing files.

The `open()` function is central to file handling. It takes the file path and a mode as arguments. Common modes include:

- `'r'` (read mode): Opens a file for reading. The file pointer is placed at the beginning of the file. If the file does not exist, it raises a `FileNotFoundError`.
- `'w'` (write mode): Opens a file for writing. If the file already exists, its contents are truncated (emptied). If the file does not exist, a new one is created. The file pointer is placed at the beginning of the file.
- `'a'` (append mode): Opens a file for appending. The file pointer is placed at the end of the file. If the file does not exist, a new one is created.

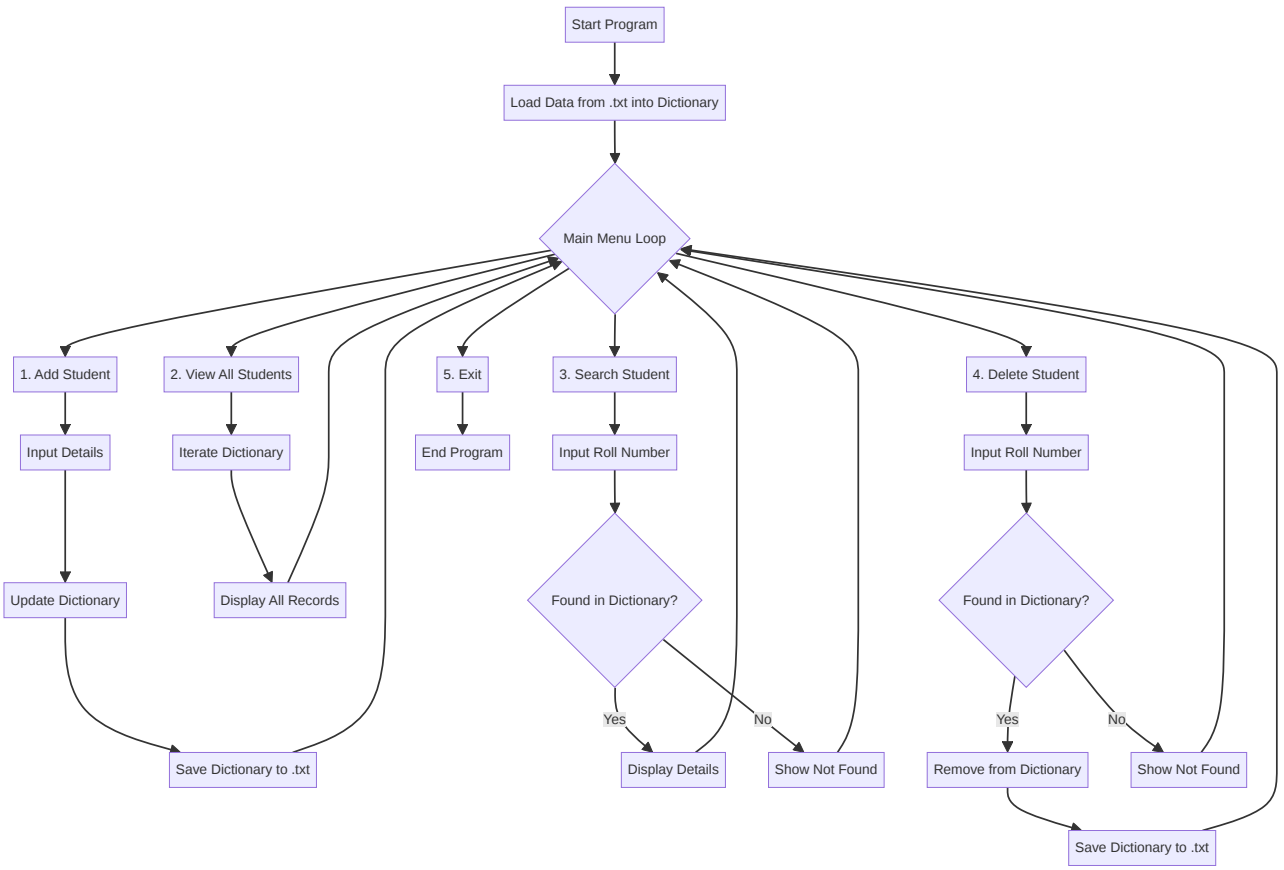
In the SARMS project, file handling is used to store and retrieve student data from a plain text file named `students.txt`. The `load_data()` function uses `'r'` mode to read existing student records from the file into the dictionary when the program starts. Conversely, the `save_data()` function uses `'w'` mode to write the current state of the student dictionary back to `students.txt`, ensuring that any additions, modifications,

or deletions are permanently stored. This mechanism guarantees that student data is not lost when the program closes and is available for subsequent sessions.

Chapter 3: System Design

The Student Academic Record Management System is designed with a simple, menu-driven interface to facilitate ease of use. The core of the system revolves around a main loop that presents the user with five distinct options: Add Student, View All Students, Search Student, Delete Student, and Exit. The data is persistently stored in a `.txt` file and dynamically loaded into a Python dictionary for in-memory operations, ensuring efficient data manipulation during runtime.

The following flowchart illustrates the system's main menu loop and the branching logic for each operation, demonstrating how data flows between the `.txt` file and the in-memory dictionary.



Data Flow Explanation:

1. **Initialization:** Upon program start, the `load_data()` function reads each line from `students.txt`. Each line, representing a student record (e.g., `RollNo,Name,Course,Grade`), is parsed, and the information is stored as a key-value pair in the `students` dictionary. The Roll Number acts as the key, and a nested dictionary containing the student's name, course, and grade forms the value.
2. **Main Menu Loop:** The program then enters a continuous loop, presenting the user with options. This loop continues until the user chooses to exit.
3. **Add Student:** When a student is added, their details are collected, and a new entry is created in the `students` dictionary. Immediately after, `save_data()` is called to write the updated dictionary back to `students.txt`, ensuring persistence.
4. **View All Students:** The system iterates through the `students` dictionary and displays all stored records in a formatted manner.
5. **Search Student:** The user provides a Roll Number, which is used as a key to look up the student in the `students` dictionary. If found, the details are displayed; otherwise, a "Student not found" message is shown.
6. **Delete Student:** Similar to adding, when a student is deleted, their entry is removed from the `students` dictionary. The `save_data()` function is then called to update `students.txt` with the modified data.
7. **Exit:** The program breaks out of the main menu loop, and the application terminates.

Chapter 4: Implementation

The core of the Student Academic Record Management System is implemented in Python, utilizing dictionaries for in-memory data storage and file handling for data persistence. Below is the complete Python code for the SARMS application.

```python
import os

DATA_FILE = "students.txt"

def load_data():
    """Load student data from a text file into a dictionary."""
    students = {}
    if not os.path.exists(DATA_FILE):
        return students

    with open(DATA_FILE, "r") as file:
        for line in file:
            # Format: RollNo,Name,Course,Grade
            parts = line.strip().split(",")
            if len(parts) == 4:
                roll_no, name, course, grade = parts
                students[roll_no] = {
                    "Name": name,
                    "Course": course,
                    "Grade": grade
                }
    return students

def save_data(students):
    """Save the dictionary data back to the text file."""
    with open(DATA_FILE, "w") as file:
        for roll_no, details in students.items():
            line = f"{roll_no},{details["Name"]},{details["Course"]},
{details["Grade"]}\n"
            file.write(line)

def add_student(students):
    roll_no = input("Enter Roll Number: ")
    if roll_no in students:
        print("Student already exists!")
        return
    name = input("Enter Name: ")
    course = input("Enter Course: ")
    grade = input("Enter Grade: ")
    students[roll_no] = {"Name": name, "Course": course, "Grade": grade}
    save_data(students)
    print("Student added successfully!")

def view_students(students):
    if not students:
```

```
        print("No records found.")
        return
    print("\n--- Student Records ---")
    print(f"{'Roll No':<10} {'Name':<20} {'Course':<15} {'Grade':<5}")
    print("-" * 55)
    for roll_no, details in students.items():
        print(f"{roll_no:<10} {details['Name']:<20} {details['Course']:<15}
{details['Grade']:<5}")

def search_student(students):
    roll_no = input("Enter Roll Number to search: ")
    if roll_no in students:
        details = students[roll_no]
        print(f"\nRecord Found:")
        print(f"Name: {details['Name']}")
        print(f"Course: {details['Course']}")
        print(f"Grade: {details['Grade']}")
    else:
        print("Student not found.")

def delete_student(students):
    roll_no = input("Enter Roll Number to delete: ")
    if roll_no in students:
        del students[roll_no]
        save_data(students)
        print("Student deleted successfully!")
    else:
        print("Student not found.")

def main():
    students = load_data()
    while True:
        print("\n=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===")
        print("1. Add Student")
        print("2. View All Students")
        print("3. Search Student")
        print("4. Delete Student")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")

        if choice == '1':
            add_student(students)
        elif choice == '2':
            view_students(students)
        elif choice == '3':
```

```
            search_student(students)
        elif choice == '4':
            delete_student(students)
        elif choice == '5':
            print("Exiting system. Goodbye!")
            break
        else:
            print("Invalid choice! Please try again.")

if __name__ == "__main__":
    main()
```

Explanation of `load_data()` and `save_data()` Functions

`load_data()` Function:

This function is responsible for reading student records from the `students.txt` file and populating the `students` dictionary at the start of the program.

1. **Initialization:** An empty dictionary `students = {}` is created to hold the data.
2. **File Existence Check:** `if not os.path.exists(DATA_FILE): return students` checks if `students.txt` exists. If not, it means there are no saved records yet, so an empty dictionary is returned.
3. **File Reading:** `with open(DATA_FILE, "r") as file:` opens the `students.txt` file in read mode (`"r"`). The `with` statement ensures the file is properly closed after its block, even if errors occur.
4. **Line Processing:** The function iterates through each `line` in the file.
    - `line.strip().split(",")` removes leading/trailing whitespace (like newline characters) and then splits the line into a list of strings using the comma (`,`) as a delimiter. This assumes each student record is stored as `RollNo,Name,Course,Grade`.
    - `if len(parts) == 4:` ensures that only properly formatted lines with four parts are processed, preventing errors from malformed data.
    - The extracted `roll_no`, `name`, `course`, and `grade` are then used to construct a nested dictionary for the student, which is added to the main `students` dictionary with `roll_no` as the key.

5. **Return Value:** Finally, the populated `students` dictionary is returned.

`save_data()` Function:

This function is responsible for writing the current state of the `students` dictionary back to the `students.txt` file, ensuring that any changes made during the program's execution are saved persistently.

1. **File Opening:** `with open(DATA_FILE, "w") as file:` opens the `students.txt` file in write mode (`"w"`). If the file exists, its contents are truncated (emptied) before writing. If it doesn't exist, a new file is created.
2. **Dictionary Iteration:** The function iterates through each `roll_no` and its corresponding `details` (the nested dictionary) in the `students` dictionary.
3. **Line Formatting and Writing:** For each student record, a string `line` is formatted using an f-string: `f"{roll_no},{details["Name"]},{details["Course"]},{details["Grade"]}\n"`. This reconstructs the `RollNo,Name,Course,Grade` format, followed by a newline character (`\n`) to ensure each record is on a new line.
4. **File Writing:** `file.write(line)` writes the formatted string to the `students.txt` file.

These two functions are critical for the system's ability to maintain data persistence, allowing the application to load previous data and save new or modified data across different sessions.

Chapter 5: Results

This chapter presents screenshots of the terminal interactions with the Student Academic Record Management System, demonstrating its core functionalities: displaying the main menu, adding a new student record, and searching for an existing student.

Main Menu

Below is the initial display of the SARMS main menu, showing the available options to the user.

```
=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5):
```

Adding a Student

This screenshot demonstrates the process of adding a new student, John Doe, with Roll Number 101, Course Computer Science, and Grade A. The system confirms the successful addition.

```
=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5): 1
Enter Roll Number: 101
Enter Name: John Doe
Enter Course: Computer Science
Enter Grade: A
Student added successfully!

=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5): 5
Exiting system. Goodbye!
```

Searching for a Student

This screenshot illustrates searching for the student with Roll Number 101. The system successfully retrieves and displays John Doe's academic details.

```
=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5): 3
Enter Roll Number to search: 101

Record Found:
Name: John Doe
Course: Computer Science
Grade: A

=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5): 5
Exiting system. Goodbye!
```

Viewing All Students

This screenshot shows the output when viewing all students after adding John Doe.

```
=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5): 2

--- Student Records ---
Roll No    Name                 Course         Grade
-------------------------------------------------------
101        John Doe             Computer Science A

=== STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM ===
1. Add Student
2. View All Students
3. Search Student
4. Delete Student
5. Exit
Enter your choice (1-5): 5
Exiting system. Goodbye!
```

Chapter 6: Conclusion

This project successfully demonstrates the application of fundamental Python programming concepts to develop a functional Student Academic Record Management System. By utilizing dictionaries for efficient in-memory data handling and file operations for persistent storage, the SARMS effectively addresses the need for a lightweight digital record-keeping solution. The system's menu-driven interface provides intuitive access to core functionalities such as adding, viewing, searching, and deleting student records, thereby streamlining administrative tasks associated with academic data management.

The implementation highlights how syllabus concepts, specifically Python dictionaries and file handling, can be directly applied to solve real-world problems. The use of a dictionary with unique roll numbers as keys ensures quick data retrieval and modification, while the `load_data()` and `save_data()` functions guarantee data integrity and persistence across program sessions. This project serves as a practical example of building a robust, albeit simple, application that can significantly improve

the efficiency and accuracy of academic record management in educational institutions. The modular design also allows for future enhancements, such as advanced search filters, data validation, or integration with more sophisticated database systems, further extending its utility.

References

[1] Python Software Foundation. (n.d.). *The Python Tutorial*. Retrieved from https://docs.python.org/3/tutorial/ [2] Real Python. (n.d.). *Python Dictionaries*. Retrieved from https://realpython.com/python-dictionaries/ [3] GeeksforGeeks. (n.d.). *File Handling in Python*. Retrieved from https://www.geeksforgeeks.org/file-handling-python/